TECHNICAL WHITEPAPER V0.1

Semantic Compiler

A domain-agnostic system that translates natural language into executable SQL by crawling warehouse topology, profiling data, and using vector-based ontology retrieval.

Node.js / Snowflake / BigQuery / Pinecone / OpenAI

Table of Contents

01 — Problem Statement

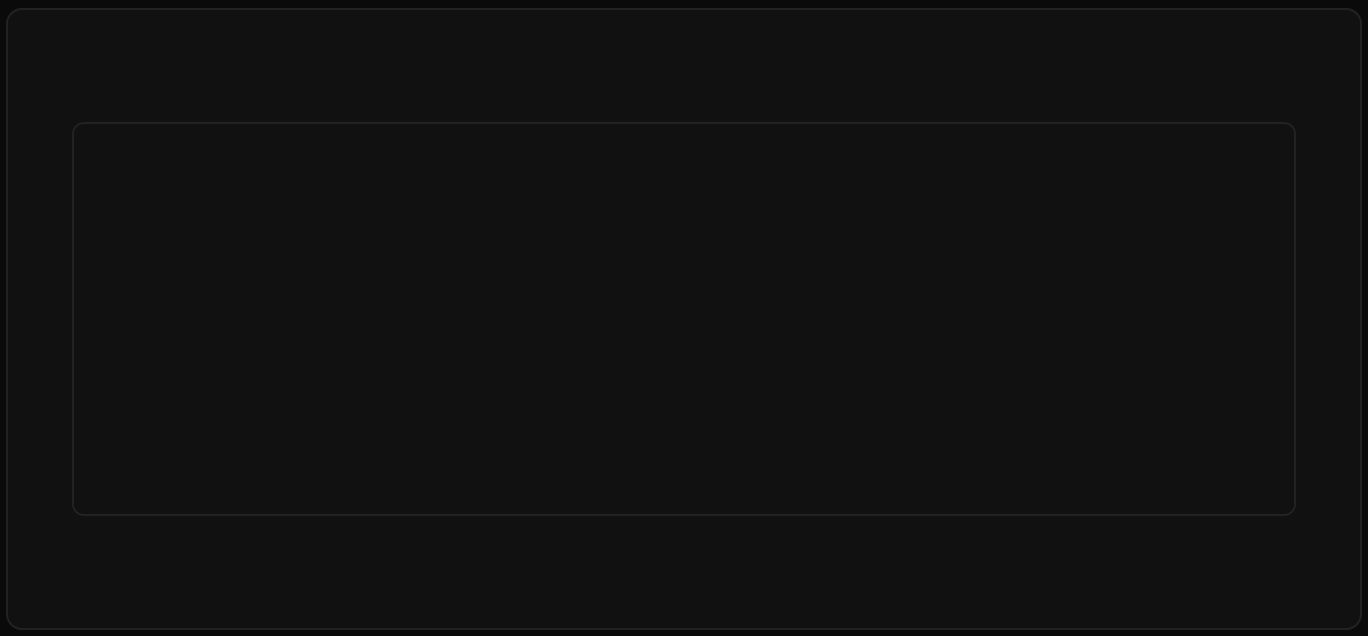
Modern data warehouses contain hundreds of tables, each with hundreds of columns using inconsistent naming conventions. Asking a business question — "What's our Facebook ROAS for Oats Overnight last 7 days?" — requires knowing which tables exist, which columns hold the revenue vs. spend data, what date ranges are available, and how to write dialect-correct SQL.

Existing text-to-SQL systems assume clean schemas with few tables. They break down against real marketing analytics warehouses where a single table can have 300+ columns from denormalized connector exports (Supermetrics, Funnel, etc.).

Constraints

The system must be **domain-agnostic** — no baked-in knowledge of marketing metrics, business logic, or column semantics. It must work against any warehouse, any schema, and derive all understanding from the data itself.

02 — System Architecture



Two phases: **offline indexing** (crawl, profile, embed) runs once per schema change. **Online query** (search, filter, compile, execute) runs per user question in 3-8 seconds.

---

03 — Pipeline Deep Dive

1

Crawl

Connects to Snowflake (key-pair auth) or BigQuery (service account). Enumerates databases/datasets, schemas, tables, views, and columns via `SHOW` commands or metadata API. Outputs `data/topology.json`.

2

Profile

For each table, runs aggregate profiling queries against the last 30 days. Numeric columns get `SUM`, `COUNT(!=0)`, `MAX`. String columns get `COUNT`, `COUNT(DISTINCT)`, `MAX`. Low-cardinality strings (distinct ≤ 20) get their exact distinct values fetched. Date columns get `MIN/MAX` range.

3

Embed

Each table's DDL (with profiling data) is converted to text and embedded via OpenAI `text-embedding-3-small` (1536 dims). Vectors are upserted to Pinecone with full DDL in metadata. Embedding text is capped at 50 columns / 16K chars; metadata stores the complete DDL.

4

Search

User query is embedded and searched against Pinecone. Results are re-ranked with keyword boost (table name matching) and dataset-aware expansion (brand mentions auto-include all tables from that dataset).

5

Filter Columns

Each matched table's 300+ columns are filtered to 30 via a three-pool system: structural columns (dates, IDs), keyword-matched columns, and data-volume columns (split 70/30 numeric/string). This prevents LLM context overflow.

6

Compile

GPT-4o receives the filtered ontology context and generates dialect-correct SQL (Snowflake or BigQuery). Column-table binding rules prevent cross-contamination. On execution failure, the error is fed back for self-correction (up to 2 retries).

7

Execute

Generated SQL runs against the warehouse. Results are formatted as a table with BigQuery date-object unwrapping and column-width capping.

---

04 — Data Profiling

The profiler is the system's eyes into the data. Without it, the LLM hallucinates column names, picks null columns, and guesses at string values. Three profiling strategies operate per table:

Numeric Profiling

```
SELECT
  SUM(col) AS col__sum,
  COUNTIF(col IS NOT NULL AND col != 0) AS col__nnz,
  MAX(col) AS col__max
FROM table
WHERE Date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
```

Produces: `Amount_Spent__Facebook_Ads (FLOAT) sum=3.7M, nonzero_rows=6029, max=11572.41`

String Profiling

```
SELECT
  COUNTIF(col IS NOT NULL) AS col__nn,
  COUNT(DISTINCT col) AS col__nuniq,
```

```
    MAX(col) AS col__sample
  FROM table
  WHERE Date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
```

Produces: `Traffic_source (STRING) nonnull_rows=10415, distinct=1, e.g. Facebook`

Low-Cardinality Enumeration

When `distinct ≤ 20`, the profiler fetches all values:

```
SELECT DISTINCT col FROM table WHERE col IS NOT NULL ORDER BY 1 LIMIT 20
```

Produces: `Customer_Sale_Type__Shopify (STRING) values: First-time | Returning`

This prevents the ”First Time” vs ”First-time” problem — the model sees exact strings and copies them verbatim.

---

05 — Retrieval & Search

```
flowchart TB
  Q[User Query] --> EMB[Embed Query]
  EMB --> SEM[Semantic Search
Pinecone top-15]
  Q --> DET[Detect Dataset
```

```
  token matching]
    DET -->|matched datasets| FILT[Filtered Search
  Pinecone metadata filter]
    SEM --> MERGE[Merge + Dedupe]
    FILT --> MERGE
    MERGE --> RANK[Keyword Boost
  + Re-rank]
    RANK --> TOP[Top-K Results]
```

Keyword Boost

TABLE SCORE

```
score = semantic_score + (matched_tokens / total_tokens) * 0.15
```

Dataset-Aware Search

When query tokens match a dataset/schema name (e.g., "oats" matches "oats_overnight"), ALL tables from that dataset are fetched via Pinecone metadata filter `{ schema: { $eq: "oats_overnight" } }`. This ensures "all oats ad spend" includes Facebook, Google, TikTok, and Shopify tables without the user naming each one.

---

06 — Semantic Compilation

The compiler is GPT-4o with a constrained system prompt. It receives filtered ontology context (30 columns per table, 5-10 tables) and generates dialect-specific SQL.

Key Prompt Constraints

CONSTRAINT	PURPOSE
Column-table binding	Prevents using a column from Table A in a SELECT from Table B
Exact name copying	Prevents hallucinating or abbreviating column names
GROUP BY enforcement	When sample data shows repeated dates, forces aggregation
Sample data awareness	Prefer columns with data; use values to infer types (counts vs dollars)
Dialect hints	Snowflake (DATEADD, ::) vs BigQuery (DATE_SUB, backticks, SAFE_DIVIDE)

Self-Correction Loop

```
flowchart LR
  C[Compile SQL] --> E[Execute]
  E -->|success| R[Results]
  E -->|error| F[Feed Error Back]
  F --> C2[Recompile]
  C2 --> E2[Execute]
  E2 -->|success| R
  E2 -->|error| F2[Feed Error Back]
  F2 --> C3[Final Attempt]
```

```
    C3 --> E3[Execute]
```

On execution failure (e.g., "Unrecognized name: spend_usd"), the failed SQL + error message are appended to the conversation. The model sees its own mistake and corrects it — typically fixing column-table binding errors on the first retry.

---

07 — Scoring & Filtering Math

Column Filter Budget (per table)

> TOTAL BUDGET
>
> `30 columns = structural + keyword_pool(15) + data_volume_pool(remaining)`

Keyword Pool Scoring

> PER COLUMN
>
> `score = keyword_matches(stem(query), stem(col_name)) + (has_data ? 3 : 0)`

Stemming strips trailing `s`, `ing`, `ed` so "customers" matches "customer" and "purchases" matches "purchase".

Data Volume Pool

SPLIT

```
70% numeric (by nonzero_rows) + 30% string (by nonnull_rows)
```

Guarantees high-activity numeric columns (spend, revenue, orders) always make the cut regardless of keyword relevance.

Embedding Text Budget

PER VECTOR

```
max(50 columns, 16K chars) for embedding; full DDL (≤39KB) in metadata
```

Columns with sample data are prioritized in the embedding text. The embedding is a lossy summary for discovery; metadata preserves the full schema for compilation.

---

08 — Key Innovations

Aggregate Profiling

Instead of sampling rows (which miss sparse data), we run SUM/COUNT aggregates. A column with 3,125 non-zero purchase values out of 10,000 rows is

Low-Cardinality Enumeration

String columns with ≤20 distinct values get their exact values listed. Eliminates "First Time" vs "First-time" guessing. The model copies the exact string.

definitively "has data" — regardless of which 100 rows you sample.

Three-Pool Column Filter

Structural (dates, IDs) + keyword-matched + data-volume columns. Ensures both query-relevant and high-activity columns survive the 300-to-30 reduction.

Dataset-Aware Search

Brand mentions trigger metadata-filtered Pinecone queries that pull ALL tables from the matching dataset. "All oats ad spend" automatically finds Facebook, Google, TikTok tables.

Error-Driven Self-Correction

Failed SQL + warehouse error message are fed back to the model. Column binding errors are fixed on first retry. Handles UNION column count mismatches, unrecognized names, and type errors.

Zero Domain Knowledge

No baked-in metric definitions, no marketing-specific synonyms. CPA, ROAS, CTR formulas are inferred from column names + aggregate statistics. Works for any warehouse domain.

---

09 — Test Results

75	21	3-8s
TABLES INDEXED	DATASETS	QUERY LATENCY

2

WAREHOUSES

Capabilities Demonstrated

QUERY TYPE	COMPLEXITY	STATUS
Single-table ROAS by day	GROUP BY + SAFE_DIVIDE	Pass
Cross-platform ad spend breakdown	3-way UNION ALL + CTE	Pass
First-time customer CPA	JOIN Shopify + ad tables, filtered by Customer_Sale_Type	Pass
Creative performance report	GROUP BY creative dimensions + aggregated metrics	Pass
Multi-table insight dashboard	UNION ALL across heterogeneous schemas with NULL padding	Pass (with retry)
Missing data detection	Correctly refuses when columns lack data	Pass (correct ERROR)
Weekly aggregation	DATE_TRUNC + GROUP BY	Pass
Top-N campaign ranking	ORDER BY + LIMIT with computed ROAS	Pass

Error Recovery

ERROR TYPE	RECOVERY
Unrecognized column name	Self-corrects by using proper JOIN

UNION column count mismatch	Adds CAST(NULL AS ...) padding
Markdown fences in SQL	Stripped automatically
Response truncation	Detected via finish_reason check

10 — Limitations & Future Work

Current Limitations

LIMITATION	IMPACT
No conversational context	Each query is independent; cannot say "now do that for Google"
No intent clarification	Ambiguous queries (no brand, no metric) pick random tables
Sparse data columns	Columns null in most recent 100 rows may get profiled as empty
30-column limit per table	Extremely wide queries may miss relevant columns
No JOIN key inference	Cross-table JOINs rely on matching column names, not FK metadata

Potential Improvements

FEATURE	APPROACH

Conversational context	Session-based message history in compile step
Intent clarification	Pre-compile LLM pass that detects ambiguity and asks for specifics
Query validation	Post-compile SQL parser that verifies column-table bindings before execution
Result caching	Hash query + ontology context for deterministic cache hits
Incremental re-indexing	Detect schema changes and re-profile only affected tables

Built with Node.js, Snowflake, BigQuery, Pinecone, OpenAI